Web 2 – Séance 11

Gestion de l’état avec useState

Le problème : une variable ne suffit pas

```
const FavoriteButton = () => {
  let isFavorite = false;

  const handleClick = () => {
    isFavorite = !isFavorite;
    console.log(isFavorite); // true, false, true ...  : la variable change bien
  };

  return (
    <button onClick={handleClick}>
      {isFavorite ? "Retirer des favoris" : "Ajouter aux favoris"}
    </button>
  );
};
```

La console montre que la variable change, mais **l'affichage reste identique**.

Comment React affiche un composant

1. React **appelle** la fonction du composant : c'est un **rendu** (*render*)
2. La fonction retourne du JSX, qui décrit l'interface à ce moment-là
3. React met à jour le DOM pour qu'il corresponde à ce JSX

Objectif → pouvoir **mettre à jour l'affichage**

- Il faut pouvoir **prévenir React** quand une valeur change
- React doit **mémoriser** cette valeur entre les rendus
- Pour calculer l'affichage, React doit **réexécuter** la fonction du composant avec la valeur mise à jour

Hook `useState`

```jsx
import { useState } from "react";

const FavoriteButton = () => {
  const [isFavorite, setIsFavorite] = useState(false);

  const handleClick = () => setIsFavorite(!isFavorite);

  return (
    <button onClick={handleClick}>
      {isFavorite ? "Retirer des favoris" : "Ajouter aux favoris"}
    </button>
  );
};
```

`useState(valeurInitiale)` retourne un tableau de deux éléments, récupérés par déstructuration :

- `isFavorite` : la valeur de l'état **pour ce rendu**
- `setIsFavorite` : la fonction qui **remplace** la valeur et demande un nouveau rendu
- Convention de nommage : `[valeur, setValeur]`

Déroulement d’un clic

```
Rendu 1 : useState(false) → isFavorite = false → bouton « Ajouter aux favoris »
            │
          clic : setIsFavorite(true)
            │   React mémorise true et planifie un nouveau rendu
            ▼
Rendu 2 : useState(false) → isFavorite = true  → bouton « Retirer des favoris »
```

- La valeur initiale n’est utilisée **qu’au premier rendu**
- Aux rendus suivants, `useState` retourne la valeur mémorisée par React
- React compare l’ancien et le nouveau JSX, et ne modifie que le texte du bouton dans le DOM

Un hook

`useState` est un **hook** : une fonction de React dont le nom commence par `use`, qui permet à un composant d'utiliser une fonctionnalité de React.

Règles des hooks :

- Uniquement dans un **composant** (ou dans un autre hook)
- Uniquement au **premier niveau** du composant
  - React mémorise les états dans l'ordre des appels de hooks
  - → Jamais dans un `if`, une boucle ou une fonction imbriquée

```
const RecipeCard = ({ recipe }: RecipeCardProps) => {
  const [isFavorite, setIsFavorite] = useState(false); // ✅ premier niveau

  if (recipe.difficulty > 3) {
    const [x, setX] = useState(0); // ❌ dans une condition
  }
  // ...
};
```

L’état est propre à chaque instance

```
const RecipeList = () ⇒ (
  <>
    <RecipeCard recipe={recipes[0]} />
    <RecipeCard recipe={recipes[1]} />
  </>
);
```

- Chaque `RecipeCard` affichée possède **son propre** état `isFavorite`
- Mettre la première en favori ne change pas la seconde
- React associe l’état à la **position du composant dans l’arbre** (et à sa `key` dans une liste)
- Si le composant disparaît de l’affichage, son état est perdu

Typer l'état

TypeScript **infère** le type à partir de la valeur initiale.

```
const [count, setCount] = useState(0);           // number
const [title, setTitle] = useState("");          // string
const [isOpen, setIsOpen] = useState(false);     // boolean
```

Il faut déclarer le type quand la valeur initiale ne suffit pas :

```
import type { Recipe } from "../models/recipe";

// Aucune recette sélectionnée au départ : sans type, TypeScript infère undefined uniquement
const [selected, setSelected] = useState<Recipe | undefined>(undefined);

// Tableau vide au départ : sans type, TypeScript infère never[]
const [favoriteIds, setFavoriteIds] = useState<number[]>([]);
```

L'état est une photo du rendu

Dans un rendu, la valeur de l'état est **fixe** : appeler le setter ne la modifie pas, il prépare le rendu suivant.

```
const Counter = () => {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1);
    console.log(count); // affiche encore l'ancienne valeur
  };

  return <button onClick={handleClick}>{count}</button>;
};
```

- `count` est une constante du rendu en cours
- La nouvelle valeur sera disponible au **prochain rendu**

Mise à jour à partir de la valeur précédente

```
const handleClick = () ⇒ {
  setCount(count + 1); // count vaut 0 : "mettre 1"
  setCount(count + 1); // count vaut toujours 0 : "mettre 1"
};
// Résultat : 1, et pas 2
```

Solution : passer une **fonction** au setter. React l'appelle avec la valeur la plus récente.

```
const handleClick = () ⇒ {
  setCount((prev) ⇒ prev + 1); // 0 → 1
  setCount((prev) ⇒ prev + 1); // 1 → 2
};
// Résultat : 2
```

Règle : quand la nouvelle valeur **dépend de l'ancienne**, utiliser la forme `setX((prev) ⇒ ...)`.

```
const toggleFavorite = () ⇒ setIsFavorite((prev) ⇒ !prev);
```

Valeurs dérivées : calculer au lieu de stocker

- **Valeurs dérivées** : valeur qui se déduit de l'état ou des props
- **Calculé pendant le rendu**, pour éviter de stocker des données redondantes

```
const ServingsSelector = ({ recipe }: { recipe: Recipe }) => {
  const [servings, setServings] = useState(recipe.servings);

  // ❌ Ne pas stocker la valeur dérivée
  const [ratio, setRatio] = useState(servings / recipe.servings);

  // ✅ Calculer à chaque rendu à partir de l'état
  const ratio = servings / recipe.servings;

  return (
    <ul>
      {recipe.ingredients.map((ingredient) => (
        <li key={ingredient.name}>
          {ingredient.quantity * ratio} {ingredient.unit} {ingredient.name}
        </li>
      ))}
    </ul>
  );
};
```

Exemple : bouton favori

```
const FavoriteButton = () => {
  const [isFavorite, setIsFavorite] = useState(false);

  return (
    <IconButton
      color="primary"
      aria-label={isFavorite ? "Retirer des favoris" : "Ajouter aux favoris"}
      onClick={() => setIsFavorite((prev) => !prev)}
    >
      {isFavorite ? <FavoriteIcon /> : <FavoriteBorderIcon />}
    </IconButton>
  );
};
```

- Le rendu dépend de l'état : l'icône est **déduite** de `isFavorite`
- On ne manipule jamais le DOM : on change l'état, React met à jour l'affichage
- `aria-label` : texte lu par les lecteurs d'écran pour un bouton sans texte

Exemple : compteur de portions

```
const ServingsCounter = () => {
  const [servings, setServings] = useState(4);

  const decrement = () => setServings((prev) => prev - 1);
  const increment = () => setServings((prev) => prev + 1);

  return (
    <Stack direction="row" spacing={1} sx={{ alignItems: "center" }}>
      <Button onClick={decrement} disabled={servings <= 1}>Moins</Button>
      <Typography>{servings} personnes</Typography>
      <Button onClick={increment}>Plus</Button>
    </Stack>
  );
};
```

- `disabled` est **déduit** de l'état

Plusieurs états dans un composant

```
const RecipeDetail = ({ recipe }: RecipeDetailProps) => {
  const [servings, setServings] = useState(recipe.servings);
  const [showSteps, setShowSteps] = useState(true);
  // ...
};
```

- Chaque appel à `useState` crée un état indépendant
- Un état par information qui change indépendamment des autres

Récapitulatif Séance 11

- **Rendu** – React appelle la fonction du composant pour obtenir le JSX à afficher
- **État** – Valeur conservée par React entre les rendus, dont la modification déclenche un nouveau rendu
- **useState** – `const [valeur, setValeur] = useState(initiale)`
- **Hooks** – Fonctions `useXxx`, appelées au premier niveau d'un composant
- **Instance** – Chaque composant affiché a son propre état
- **Typage** – Inféré, ou explicite : `useState<Recipe | undefined>(undefined)`
- **Photo du rendu** – La valeur de l'état ne change pas pendant un rendu
- **Mise à jour fonctionnelle** – `setX((prev) ⇒ ...)` quand la valeur dépend de l'ancienne
- **Valeurs dérivées** – Calculées pendant le rendu, jamais stockées dans un état

Prochaine séance : Séance 12 – Événements et formulaires

Exercice filé S11

1. Créez un composant `FavoriteButton` (état `isFavorite`, icônes `Favorite` / `FavoriteBorder`) et ajoutez-le dans les `CardActions` de `RecipeCard`
2. Vérifiez que chaque carte a son propre état favori
3. Dans `RecipeDetail`, ajoutez un sélecteur du nombre de portions (boutons « − » et « + », minimum 1), initialisé au nombre de portions de la recette
4. Les quantités des ingrédients affichées doivent être recalculées selon le nombre de portions choisi
5. Arrondissez les quantités recalculées à une décimale au maximum
6. **Optionnel** : ajoutez un bouton qui masque ou affiche la liste des étapes

Exercice complémentaire EC05

1. Créez un projet `EC05` avec Vite + React + TypeScript
2. **Compteur borné** : un compteur avec des boutons « −1 », « +1 » et « Remise à zéro » ; la valeur reste entre 0 et 10 et les boutons inutilisables sont désactivés
3. **Thème** : un bouton bascule entre un thème clair et un thème sombre, appliqué au style d'un conteneur (couleur de fond et de texte) ; le texte du bouton indique le thème vers lequel on bascule
4. **Vote** : deux boutons « Pour » et « Contre » avec un compteur chacun, le total des votes et le pourcentage de votes « Pour »
5. Chaque exercice est un composant séparé, affiché dans `App.tsx`